Subject

Object Oriented programming

and

Computer Graphics

Unit-II

Introduction to Classes and Objects and Arrays

Contents

- **Introduction to Classes and Objects:**
  - Defining a Class, Field declaration, method declaration and definition,
  - instantiating an object of a Class,
  - Accessing class members,
  - declaring methods with multiple parameters,
  - argument passing, object as a parameter, returning objects, assigning object reference variables,
  - set methods and get methods,
  - constructors, this keyword, static methods, scope of declaration, method overloading and Java API packages.

- **Arrays:**
  - Declaring and creating arrays in Java,
  - Examples using arrays, passing arrays to methods,
  - Multidimensional arrays,
  - Variable-length argument lists using command-line arguments.

- **Managing I/O**: Streams: Byte Streams and Character Streams, Predefined Streams, Reading console Input, Writing Console Output, Print Writer class.

- **Case Studies:** Demonstrate an interactive Banking/Library management system using class, objects, array of objects

Introduction to Classes and Objects:

- **What is a Class?**
- A class is a blueprint for creating objects.
- It defines fields (variables) and methods (functions).
- Class does not occupy memory.

- **Class Declaration in Java**

```
access_modifier class <class_name>
{
data member;
method;
constructor;
nested class;
interface;
}
```

Example of class

```
class Student {

   // data member (also instance variable)
   String n;
  int age;
   // member function (also instance variable)
    void displayInfo() {
       System.out.println("Name: " + n);
       System.out.println("Age: " + age);
     }
   public static void main(String args[]) {

       // creating an object of class Student
       Student s1 = new Student();

       s1.n = "Alice";          // Accessing fields
       s1.age = 20;
       s1.displayInfo();          // Calling method
       }
}
```

Java Objects

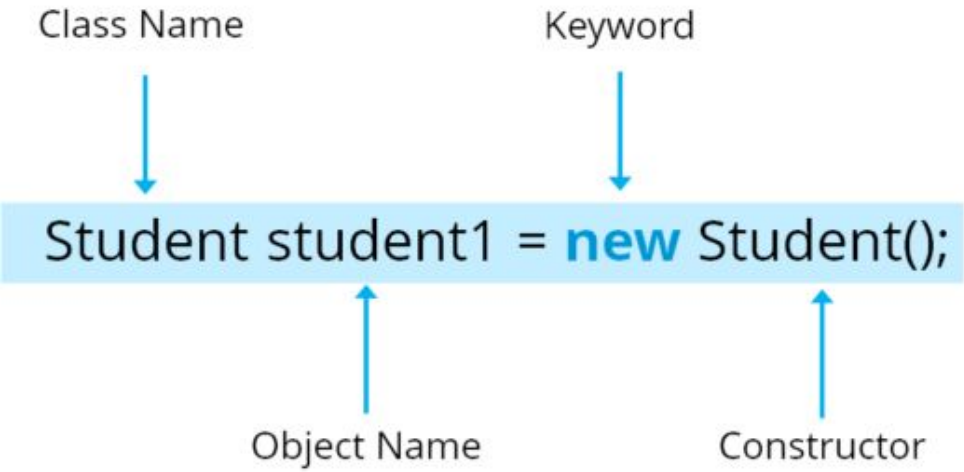
- Objects are the instances of a class that are created to use the attributes and methods of a class.
- An object consists of:

1. **State**: It is represented by attributes of an object. It also reflects the properties of an object.
2. **Behavior**: It is represented by the methods of an object. It also reflects the response of an object with other objects.
3. **Identity**: It gives a unique name to an object and enables one object to interact with other objects.

Instantiating an Object

- Use the new keyword to create an object.

```
class Student{
        public static void main(String args[]) {
            // creating an object of class Student
                Student s1 = new Student();
        }
}
```



Accessing class members

- Use dot (.) operator to access fields and methods.
- Syntax:

1) objectName.varibaleName;

```
s1.name = "Alice";
s1.age = 20;
```

2) objectName.methodName();

```
s1.displayInfo();  // Calling method
```

DECLARING METHODS WITH MULTIPLE PARAMETERS

Syntax:

accessModifier returnType methodName(dataType1 parameter1, dataType2 parameter2, ..., dataTyepN parameterN) {

// method body

}

Example:

```
public class Main {
  void myMethod(String fname) {
    System.out.println(fname + " Refsnes");
  }

  public static void main(String[] args) {
    Main obj=new Main();
    obj.myMethod("Liam");
  }

}
```

Argument passing in method:

- In Java, arguments are passed to methods by including them within the parentheses during a method call.
- These arguments correspond to the parameters defined in the method's declaration.

```
public class MyClass {

    // Method declaration with parameters
    public static void myMethod(String name, int age) {
        System.out.println(name + " is " + age + " years old.");
    }

    public static void main(String[] args) {
        // Method call with arguments
        myMethod("Alice", 30);
        myMethod("Bob", 25);
    }
}
```

PASSING OBJECT AS A PARAMETER

- In Java, objects can be passed as parameters to methods, enabling manipulation or access of the object's data within the method.
- This is achieved by defining the method parameter with the class name as its data type.

```
public class MyClass {
     public void processStudent(Student s) {
        // Method body
   }
}

public static void main(String args[]){
Student myStudent = new Student("Alice");
   processStudent(myStudent);
}
```

Behavior with Object Parameters:

Pass-by-Value (of Reference):

In Java, all arguments, including objects, are passed by value. When an object is passed, the value of the object's reference is passed to the method. This means the method receives a copy of the reference, not a copy of the object itself.

Modifying Object State:

If the method modifies the state (instance variables) of the object referenced by the passed parameter, these changes will be reflected in the original object outside the method, because both the original reference and the parameter's reference point to the same object in memory.

Reassigning the Reference:

If the method attempts to reassign the parameter to a new object, this change will not affect the original reference outside the method. The parameter's reference will point to the new object within the method's scope, but the original reference will still point to the original object.

```
class MyObject {
    int value;

    MyObject(int value) {
        this.value = value;
    }
}

public class ObjectParameterExample {

    public static void modifyObject(MyObject obj) {
        obj.value = 100; // Modifies the state of the original object
    }

    public static void reassignObject(MyObject obj) {
        obj = new MyObject(200); // Reassigns the local parameter, not the original
   reference
    }

    public static void main(String[] args) {
        MyObject originalObj = new MyObject(10);
        System.out.println("Original value before modification: " + originalObj.value); //
   Output: 10

        modifyObject(originalObj);
        System.out.println("Original value after modification: " + originalObj.value); //
   Output: 100

        reassignObject(originalObj);
        System.out.println("Original value after reassignment attempt: " +
   originalObj.value); // Output: 100 (not 200)
    }
}
```

RETURNING OBJECTS FROM METHOD:

Use the return keyword: Inside the method, use the return keyword followed by an instance of the object being returned.

```
// Java Program to Demonstrate Returning of Objects

// Class 1
class Demo {
    int a;

    // Constructor
    Demo(int i) { a = i; }

    // Method returns an object
    Demo incrByTen()
    {
        Demo temp = new Demo(a + 10);
        return temp;
    }
}
```

```
public class GFG {


    public static void main(String args[])
    {


        Demo ob1 = new Demo(2);
         Demo ob2;

        ob2 = ob1.incrByTen();

        System.out.println("ob1.a: " + ob1.a);
        System.out.println("ob2.a: " + ob2.a);
    }
}
```

SET METHODS AND GET METHODS

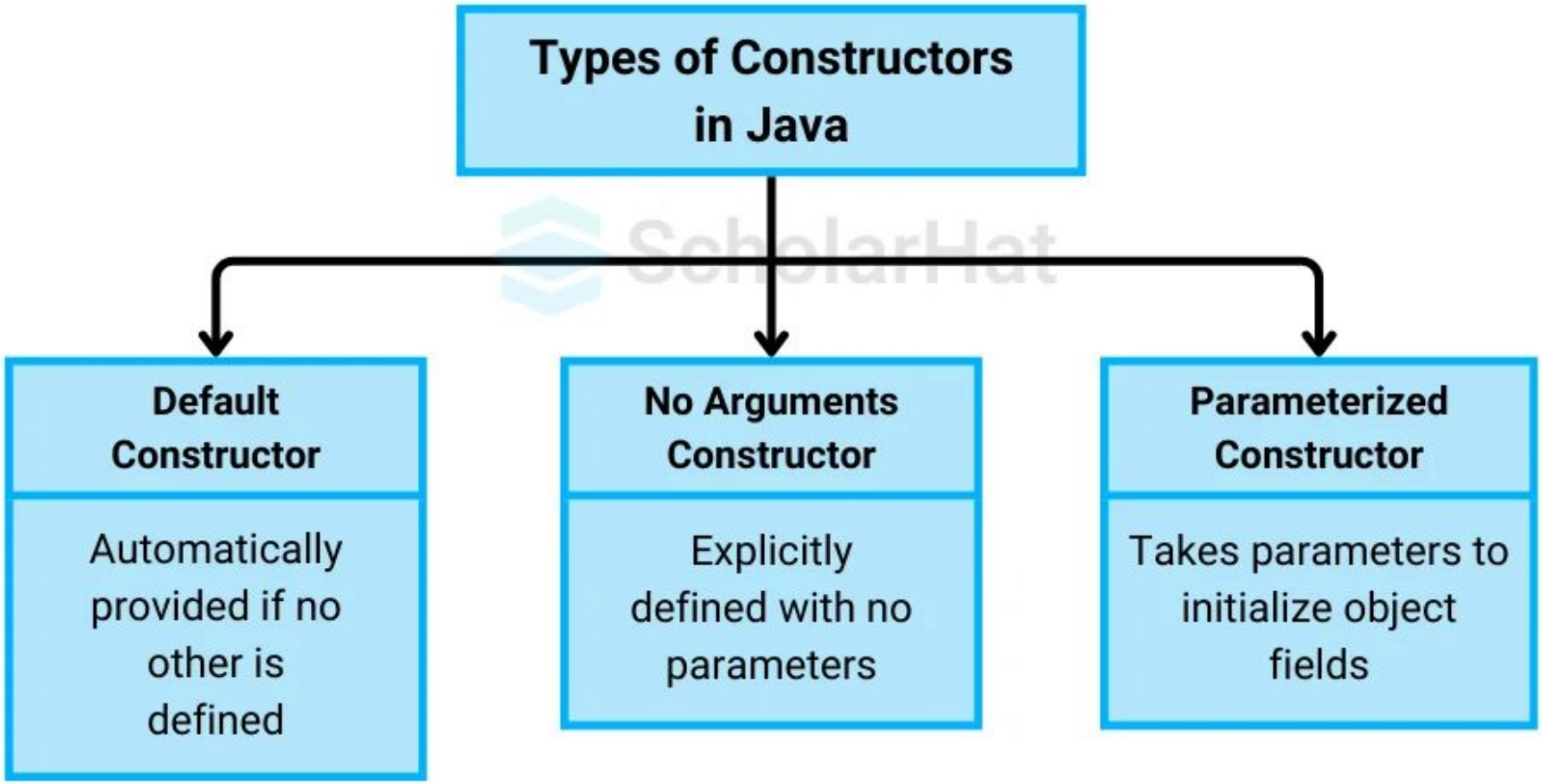
```
public class Person {
  private String name;

 // Getter
  public String getName() {
    return name;
  }

  // Setter
  public void setName(String newName) {
    this.name = newName;
  }
}
```

CONSTRUCTORS

- A constructor in Java is a special method that is used to initialize objects.
- The constructor is called when an object of a class is created.
- It can be used to set initial values for object attributes.
- Note that the constructor name must match the class name, and it cannot have a return type (like void).
- Also note that the constructor is called when the object is created.
- All classes have constructors by default: if you do not create a class constructor yourself, Java creates one for you(i.e Default constructor).

```
// Create a Demo class
public class Demo {
  int x;  // Create a class attribute

  // Create a class constructor for the Main class
  public Demo() {
    x = 5;  // Set the initial value for the class
 attribute x
  }

  public static void main(String[] args) {
    Demo myObj = new Demo(); // Create an object of class
                                Demo (This will call the constructor)

    System.out.println(myObj.x); // Print the value of x
  }
}


// Outputs 5
```

TYPES OF CONSTRUCTOR IN JAVA



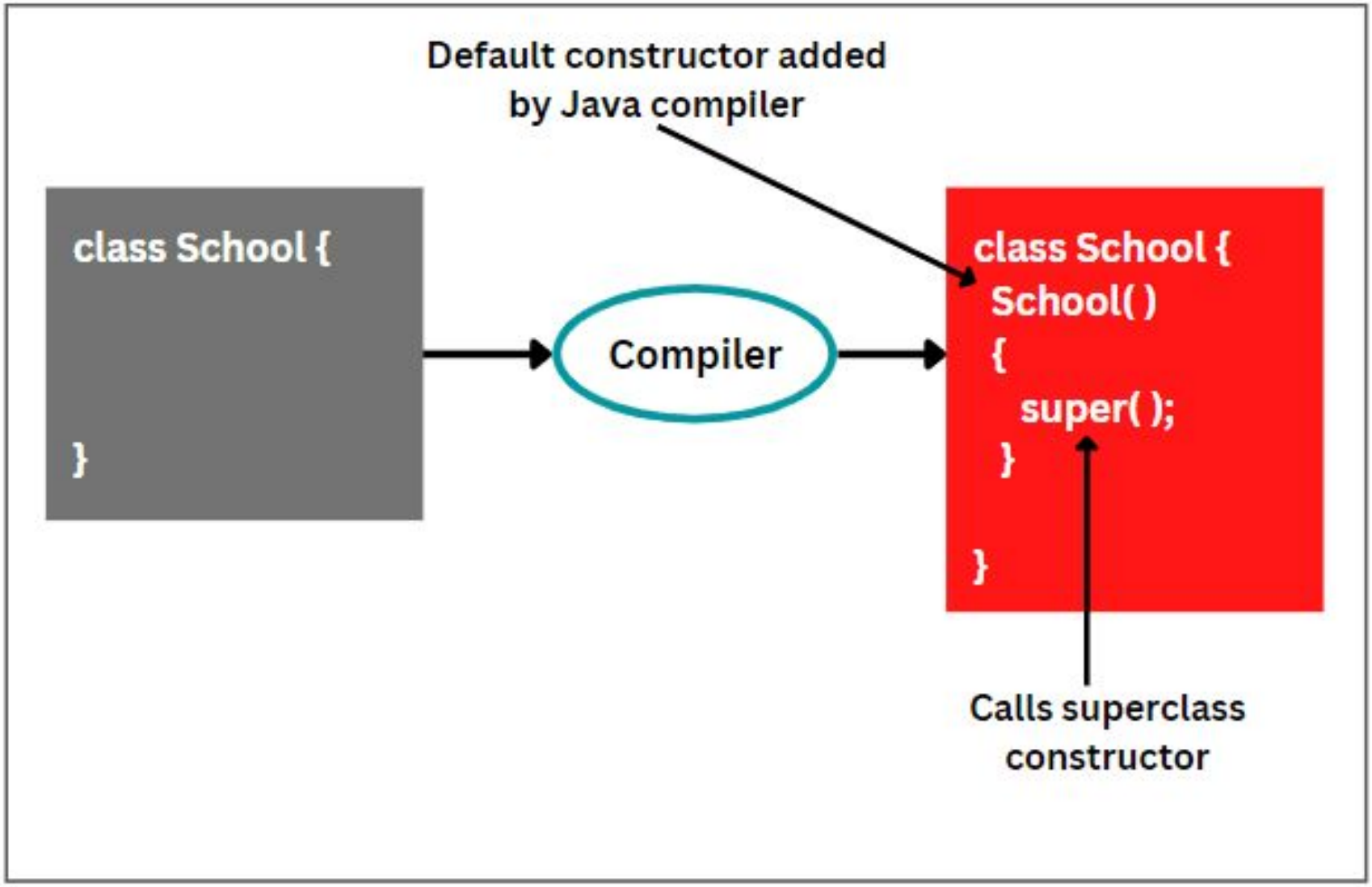
1) Default Constructor in Java:

- If no constructor is explicitly defined within a class, the Java compiler automatically provides a default, no-argument constructor during compilation.

- This constructor initializes instance variables with their default values (e.g., 0 for numeric types, null for reference types, false for booleans).

- It has no parameters and an empty body.

```
public class MyClass {
  int myInt;
  String myString;

  public void display() {
    System.out.println("myInt = " + myInt);
    System.out.println("myString = " + myString);
  }
  public static void main(String[] args) {
    MyClass obj = new MyClass();
    obj.display();
  }
}
```


Default constructor added
by Java compiler
class School {
}
Compiler
class School {
School( )
{
super( );
}
}
Calls superclass
constructor

2) No argument constructor in Java

- In Java, no argument constructors allow objects to be initialized with no data passed into them.
- A no-argument constructor is usually used when no information is needed to initialize the object.
- This is a constructor explicitly defined by the programmer that takes no arguments.
- Unlike the default constructor, the programmer can define a specific body for initialization logic within this constructor.

```
public class Person {
    private String name;
    private int age;

        public Person() {
         this.name = "John Doe";
         this.age = 30;
    }

    public void sayHello() {
        System.out.println("Hello, my name is " + this.name + " and I am " + this.age + "
years old.");
    }

        public static void main(String[] args) {
            Person john = new Person();

              john.sayHello();
    }
}
```

3) Parameterized constructor in Java

- This type of constructor takes one or more parameters.
- It allows for the initialization of object fields with specific values provided at the time of object creation.
- Parameterized constructors are commonly used to set initial states for objects based on external input.

```
class  Demo {

  String languages;

  // constructor accepting single value
  Demo(String lang) {
    languages = lang;
    System.out.println(languages + " Programming Language");
  }

  public static void main(String[] args) {

    // call constructor by passing a single value
    Demo obj1 = new  Demo("Java");
    Demo obj2 = new Demo("Python");
    Demo obj3 = new Demo("C");
  }
}
```

THIS KEYWORD

The this keyword refers to the current object in a method or constructor.

The most common use of the this keyword is to eliminate the confusion between class attributes and parameters with the same name (because a class attribute is shadowed by a method or constructor parameter).

Use of this Keyword:

1. To refer to the current class instance variable.
2. To invoke the current class method.
3. To invoke the current class constructor.
4. To pass the current object as a parameter to a method.
5. To return the current object from a method.
6. To pass the current object as a parameter to a constructor.
7. To access the outer class instance from an inner class.

Example 1: Referencing Instance Variables

```java
public class ThisExample {
    int a;
    int b;

    ThisExample(int a, int b) {
        this.a = a;
        this.b = b;
    }

    void display() {
        System.out.println("a: " + this.a + ", b: " + this.b);
    }

    public static void main(String[] args) {
        ThisExample obj = new ThisExample(10, 20);
        obj.display();
    }
}
```

Example 2: Invoking a Method

```java
public class ThisMethodExample {
    void display() {
        System.out.println("Hello, World!");
    }

    void invokeDisplay() {
        this.display();
    }

    public static void main(String[] args) {
        ThisMethodExample obj = new ThisMethodExample();
        obj.invokeDisplay();
    }
}
```

Example 3: Invoking a Constructor

```java
public class ThisConstructorExample {
    int a;
    int b;

    ThisConstructorExample() {
        this(10, 20);
    }

    ThisConstructorExample(int a, int b) {
        this.a = a;
        this.b = b;
    }

    void display() {
        System.out.println("a: " + a + ", b: " + b);
    }

    public static void main(String[] args) {
        ThisConstructorExample obj = new ThisConstructorExample();
        obj.display();
    }
}
```

4. To pass the current object as a parameter to a method.

```
class Example {
    void display(Example obj) {
        System.out.println("Object passed: " + obj);
    }

    void callDisplay() {
        display(this); // Passes the current object
    }
}
```

5. Returning the Current Object

```java
class Example {
    int num;

    Example setNum(int num) {
        this.num = num;
        return this; // Returns the current object
    }

    void display() {
        System.out.println("Number: " + num);
    }
}
```

Constructor overloading

- Constructor overloading in Java allows a class to have multiple constructors, each with a different signature.

- The signature of a constructor is determined by the **number, type, and order of its parameters**. This feature provides flexibility in object creation, enabling you to initialize objects in various ways depending on the specific needs.

- **Automatic Selection:** When an object is created using the new keyword, Java automatically determines which constructor to invoke based on the arguments provided during object instantiation.

Key aspects of constructor overloading:

Multiple Constructors:

A single class can define multiple constructors.

Different Signatures:

Each overloaded constructor must have a unique parameter list. This difference can be in:

- **Number of parameters:** e.g., one constructor takes one parameter, another takes two.
- **Type of parameters:** e.g., one constructor takes an `int`, another takes a `String`.
- **Order of parameters:** e.g., one constructor takes `(int, String)`, another takes `(String, int)`.

```java
public class MyClass {
    private int value1;
    private String value2;

    // Default constructor
    public MyClass() {
        this.value1 = 0;
        this.value2 = "Default";
    }

    // Constructor with one integer parameter
    public MyClass(int value1) {
        this.value1 = value1;
        this.value2 = "Default";
    }

    // Constructor with one string parameter
    public MyClass(String value2) {
        this.value1 = 0;
        this.value2 = value2;
    }
```

```
public void display() {
        System.out.println("Value 1: " + value1 + ", Value 2: " +
value2);
    }

    public static void main(String[] args) {
        MyClass obj1 = new MyClass(); // Calls default
                                        constructor
        MyClass obj2 = new MyClass(10); // Calls constructor with
                                          int parameter
        MyClass obj3 = new MyClass("Custom"); // Calls
                                            constructor with String

        obj1.display(); // Output: Value 1: 0, Value 2: Default
        obj2.display(); // Output: Value 1: 10, Value 2: Default
        obj3.display(); // Output: Value 1: 0, Value 2: Custom
    }
}
```

Java Constructor	Java Method
Constructor is used to initialize the state of an object.	Method is used to expose behavior of an object.
Constructor must not have return type.	Method must have return type.
Constructor is invoked implicitly.	Method is invoked explicitly.
The java compiler provides a default constructor if you don't have any constructor.	Method is not provided by compiler in any case.
Constructor name must be same as the class name.	Method name may or may not be same as class name.

STATIC METHODS

In Java, **static methods** are methods that belong to the class rather than any specific instance of the class.

They are defined using the **static** keyword and can be called without creating an object of the class.

Key Features of Static Methods:

- **Class-Level Association**: Static methods are associated with the class, not with any object.
- **Direct Access**: They can be called directly using the class name.
- **No Instance Variables**: Static methods cannot access instance variables or instance methods directly. They can only access static variables and other static methods.
- **Utility Methods**: Often used for utility or helper methods (e.g., Math.pow()).

```java
class Example {
    // Static method
    static void displayMessage() {
        System.out.println("This is a static method.");
    }

    public static void main(String[] args) {
        // Calling static method directly using class name
        Example.displayMessage();
    }
}
```

SCOPE OF DECLARATION IN JAVA

In Java, the **scope of a variable** refers to the region of the program where the variable is accessible.

1. Class-Level Scope (Instance and Static Variables)

- **Instance Variables**: Declared inside a class but outside any method, constructor, or block. They belong to an instance of the class and are accessible as long as the object exists.

Example:

```
public class Example {
    int instanceVar = 10; // Instance variable
}
```

- **Static Variables**: Declared with the static keyword. They belong to the class rather than any specific instance and are shared among all objects of the class.

- **Example**:

```
public class Example {
    static int staticVar = 20; // Static variable
}
```

2. Method-Level Scope (Local Variables)

- Declared inside a method and accessible only within that method. They are created when the method is called and destroyed when the method exits.

- **Example:**

```java
public void display() {
    int localVar = 30; // Local variable
    System.out.println(localVar);
}
```

3. Block-Level Scope

- Declared inside a block of code (e.g., within {} such as loops, if statements). These variables are accessible only within that block.

- **Example:**

```java
public void check() {
    if (true) {
        int blockVar = 40; // Block variable
        System.out.println(blockVar);
    }
    // blockVar is not accessible here
}
```

METHOD OVERLOADING

- **Method Overloading** in Java allows a class to have multiple methods with the same name but different parameter lists.
- It is a feature of **Compile-time Polymorphism** (also known as **Static Polymorphism** or **Early Binding**) because the method to be executed is determined at compile time.

- **Ways of method overloading by changing:**
- **The number of parameters**
- **The types of parameters**
- **The order of parameters**

1) **Different Parameter Lists**:
   - Methods must differ in the **number** or **type** of parameters.
   - Example:

```
void display(int a) { }
void display(String b) { }
```

2) **Return Type Doesn't Matter**:

- Overloading cannot be achieved by changing only the return type of methods.
- Example (Invalid):

```
int display() { }
void display() { } // Error: Duplicate method
```

```java
class Calculator {
    // Method to add two integers
    public int add(int a, int b) {
        return a + b;
    }

    // Method to add three integers (overloaded)
    public int add(int a, int b, int c) {
        return a + b + c;
    }

    // Method to add two doubles (overloaded)
    public double add(double a, double b) {
        return a + b;
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        Calculator calculator = new Calculator(); // Create a Calculator object

        // Calling the overloaded add methods
        int sumTwoInts = calculator.add(5, 3); // Calls add(int a, int b)
        System.out.println("Sum of two integers: " + sumTwoInts);

        int sumThreeInts = calculator.add(5, 3, 2); // Calls add(int a, int b, int c)
        System.out.println("Sum of three integers: " + sumThreeInts);

        double sumTwoDoubles = calculator.add(2.5, 3.7); // Calls add(double a, double b)
        System.out.println("Sum of two doubles: " + sumTwoDoubles);
    }
}
```

Benefits of Method Overloading:

- **Code Reusability**: Same method name can handle different types of inputs.
- **Improved Readability**: Reduces the need for multiple method names for similar operations.
- **Flexibility**: Makes the program more adaptable to changes.

Java API packages

- Java API packages are collections of related classes and interfaces that provide a wide range of pre-built functionalities for Java development.

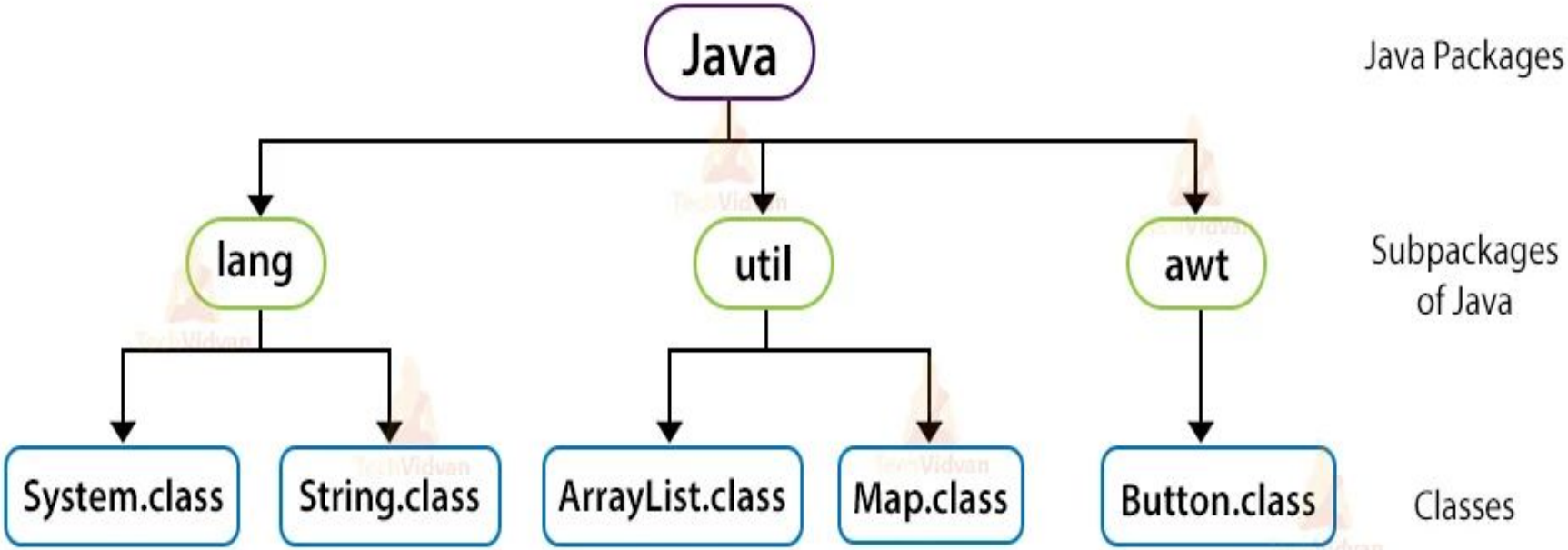
- Think of it as a **folder in a file directory**.

- We use packages to avoid name conflicts, and to write a better maintainable code.

- Packages are divided into two categories:

1. **Built-in Packages** (packages from the Java API)
2. **User-defined Packages** (create your own packages)

Built-in Packages in Java



To use a class or a package from the library, you need to use the import keyword:

Syntax

```
import package.name.Class;    // Import a single class
import package.name.*;    // Import the whole package
```

2) User-defined Packages:

- Developers can create their own packages to organize their custom classes and interfaces.
- This is done by including a package statement at the beginning of each source file that belongs to the package.
- Conventionally, package names are in reverse domain notation (e.g., com.example.myapp).

- To create your own package, you need to understand that Java uses a file system directory to store them. Just like folders on your computer:
- Example:

```
└── root
    └── mypack
        └── MyPackageClass.java
```

- To create a package, use the package keyword:

MyPackageClass.java

```
package mypack;
class MyPackageClass {
  public static void main(String[] args) {
    System.out.println("This is my package!");
  }

}
```

Save the file as MyPackageClass.java, and compile it:

```
C:\Users\Your Name>javac MyPackageClass.java
```

Then compile the package:

```
C:\Users\Your Name>javac -d . MyPackageClass.java
```

- This forces the compiler to create the "mypack" package.
- The -d keyword specifies the destination for where to save the class file. You can use any directory name, like c:/user (windows), or, if you want to keep the package within the same directory, you can use the dot sign ".", like in the example above.

Note: The package name should be written in lower case to avoid conflict with class names.

- When we compiled the package in the example above, a new folder was created, called "mypack".

- To run the MyPackageClass.java file, write the following:

```
C:\Users\Your Name>java mypack.MyPackageClass
```

- The output will be:

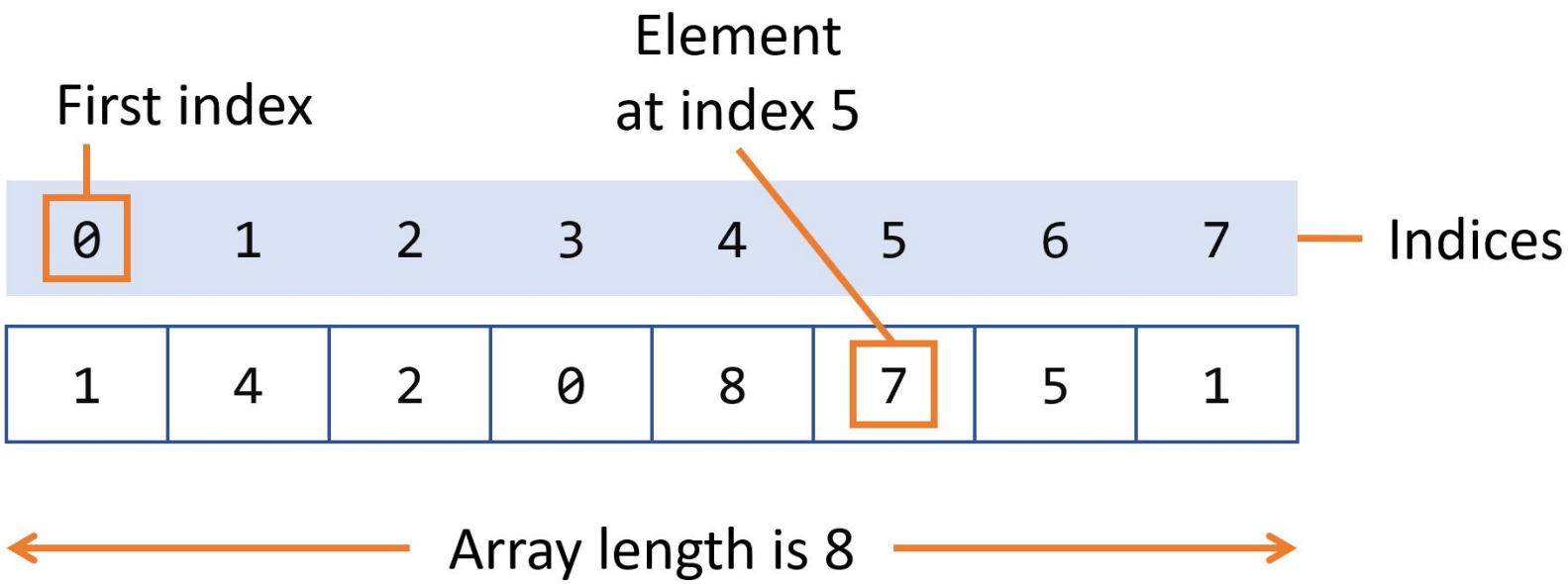
```
This is my package!
```

Arrays:

- Declaring and creating arrays in Java,
- Examples using arrays,
- Passing arrays to methods,
- Multidimensional arrays,
- Variable-length argument lists, using command-line arguments.

ARRAYS:

- Arrays in Java are fundamental data structures used to store a fixed-size, sequential collection of elements of the same data type.
- They are objects in Java and are dynamically allocated.



Key Characteristics of Arrays in Java:

- **Fixed Size:**

  Once an array is created, its size (number of elements it can hold) is fixed and cannot be changed.

- **Homogeneous Elements:**

  An array can only store elements of a single, specified data type (e.g., int, String, or custom objects).

- **Zero-Based Indexing:**

  Elements in an array are accessed using an index, which starts from 0 for the first element, 1 for the second, and so on.

- **Objects in Java:**
  Arrays are treated as objects in Java, meaning they can be assigned to variables, passed as method parameters, and returned from methods.
- **Memory Allocation:**
  For primitive data types, array elements are stored in contiguous memory locations. For object types, references to the actual objects are stored contiguously, while the objects themselves may reside in different memory locations on the heap.
- **length Property:**
  The size of an array can be retrieved using the length property (e.g., arrayName.length).

Declaring and Initializing Arrays:

Arrays can be declared and initialized in several ways:

1) Declaration and Allocation.

```
dataType[] arrayName = new dataType[size];
```

Example:

```
int[] numbers = new int[5];
```

2) Declaration, Allocation, and Initialization:

```
dataType[] arrayName = {value1, value2,
value3, ...};
```

Example:

```
String[] names = {"Alice", "Bob", "Charlie"};
```

Accessing Array Elements:

Individual elements of an array are accessed using their index within square brackets:

arrayName[index]

Example:

```
int[] numbers = {10, 20, 30};

System.out.println(numbers[0]); // Output: 10

numbers[1] = 25; // Modifies the second element
```

array variable name

Type of array

element type

length

```
int[]  myInts = new int [10];

int[] intValues = {1, 5, 18, 3, 7, 9, 23, 5, 11, 2};
```

List of Initialization Values

Type of array

array variable name

Iterating Through Arrays:

Arrays can be iterated using various loops, such as the for loop or the enhanced for-each loop:

```
// Using a traditional for loop

for (int i = 0; i < numbers.length; i++) {

   System.out.println(numbers[i]);

}

// Using an enhanced for-each loop

for (int number : numbers) {

   System.out.println(number);

}
```

Array Programs

```
public class Main {
    public static void main(String[] args)
    {

        // initializing array
        int[] arr = { 1, 2, 3, 4, 5 };

        // size of array
        int n = arr.length;

        // traversing array
        for (int i = 0; i < n; i++)
            System.out.print(arr[i] + " ");
    }
}
```

Output

```
1 2 3 4 5
```

Types of Arrays in Java

1. Single-Dimensional Arrays

```
int[] singleDimArray = {1, 2, 3, 4, 5};
```

2. Multi-Dimensional Arrays

```
int[][] multiDimArray = {{1, 2, 3},
                         {4, 5, 6},
                         {7, 8, 9} };
```

Arrays of Objects in Java

An array of objects is created like an array of primitive-type data items in the following way.

Syntax:

Method 1:

ObjectType[] arrName;

Method 2:

ObjectType arrName[];

Example of Arrays of Objects

```
class Student {
    public int roll_no;
    public String name;

    Student(int roll_no, String name){
        this.roll_no = roll_no;
        this.name = name;
    }
}
public class Main {
    public static void main(String[] args){

        // declares an Array of Student
        Student[] arr;
```

```
// allocating memory for 5 objects of type Student.
        arr = new Student[5];

        // initialize the elements of the array
        arr[0] = new Student(1, "aman");
        arr[1] = new Student(2, "vaibhav");
        arr[2] = new Student(3, "shikar");
        arr[3] = new Student(4, "dharmesh");
        arr[4] = new Student(5, "mohit");

// accessing the elements of the specified array
        for (int i = 0; i < arr.length; i++)
            System.out.println("Element at " + i + " : { "
                                + arr[i].roll_no + " "
                                + arr[i].name+" }");
    }
}
```

Output

Element at 0 : { 1 aman }

Element at 1 : { 2 vaibhav }

Element at 2 : { 3 shikar }

Element at 3 : { 4 dharmesh }

Element at 4 : { 5 mohit }

Note:

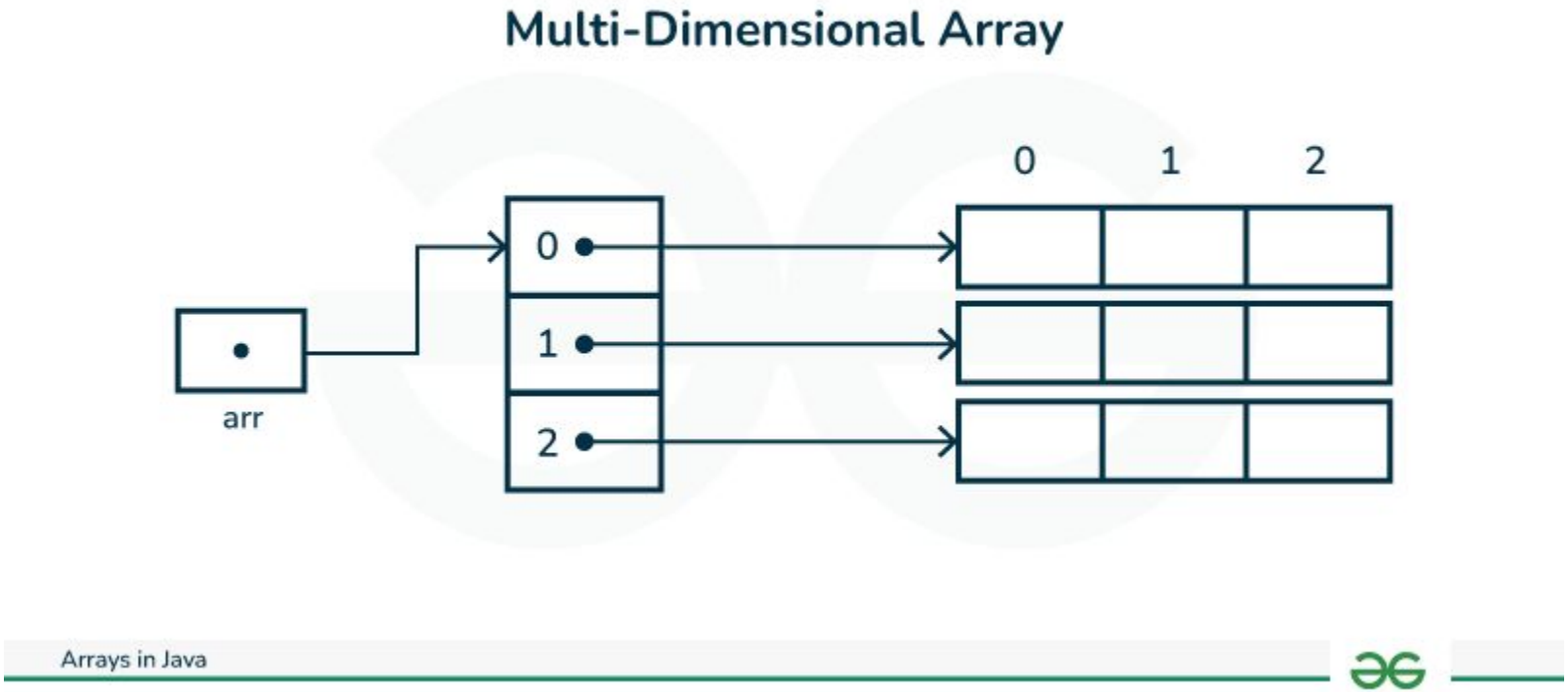
What happens if we try to access elements outside the array size?

*JVM throws **ArrayIndexOutOfBoundsException** to indicate that the array has been accessed with an illegal index. The index is either negative or greater than or equal to the size of an array.*

Multidimensional Arrays in Java

- Multidimensional arrays are **arrays of arrays** with each element of the array holding the reference of other arrays.
- A multidimensional array is created by appending one set of square brackets ([]) per dimension.
- **Syntax:**
- There are 2 methods to declare Java Multidimensional Arrays as mentioned below:

```
// Method 1
datatype [][] arrayrefvariable;

// Method 2
datatype arrayrefvariable[][];
```



Declaration:

```
// 2D array or matrix
int[][] intArray = new int[10][20];

// 3D array
int[][][] intArray = new int[10][20][10];
```

Java Multidimensional Arrays Examples

```
import java.io.*;

class GFG {
   public static void main(String[] args){

      // Two Dimensional Array
     // Declared and Initialized
     int[][] arr = new int[3][3];

      // Number of Rows
      System.out.println("Rows : " + arr.length);

      // Number of Columns
      System.out.println("Columns : " + arr[0].length);
   }
}
```

Program 2:

```
public class multiDimensional {
    // main function
  public static void main(String args[])
  {
     // declaring and initializing 2D array
     int arr[][] = { { 2, 7, 9 }, { 3, 6, 1 }, { 7, 4, 2 } };

     // printing 2D array
     for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++)
           System.out.print(arr[i][j] + " ");

        System.out.println();
     }
  }
}
```

Output

```
2 7 9
3 6 1
7 4 2
```

Passing Arrays to Methods

Like variables, we can also pass arrays to methods.

Program:

```
public class Test {
    // Driver method
    public static void main(String args[])
    {
        int arr[] = { 3, 1, 2, 5, 4 };

        // passing array to method m1
        sum(arr);
    }
```

```
public static void sum(int[] arr)
    {
        // getting sum of array values
        int sum = 0;

        for (int i = 0; i < arr.length; i++)
            sum += arr[i];

        System.out.println("sum of array values : " + sum);
    }
}
```

Output

```
sum of array values : 15
```

Returning Arrays from Methods

- As usual, a method can also return an array.

```
class Test {
    // Driver method
    public static void main(String args[])
    {
        int arr[] = m1();

        for (int i = 0; i < arr.length; i++)
            System.out.print(arr[i] + " ");
    }

    public static int[] m1()
    {
        // returning  array
        return new int[] { 1, 2, 3 };
    }
}
```

Output

```
1 2 3
```

Variable Arguments (Varargs) in Java

- In Java, Variable Arguments (Varargs) write methods that can take any number of inputs, which simply means we do not have to create more methods for different numbers of parameters.

- This feature was introduced in Java 5 to simplify method calls and reduce the need for method overloading when the number of arguments might vary.

```java
public class VarargsExample {

    public static void printNumbers(int... numbers) {
        System.out.print("Received numbers: ");
        for (int num : numbers) {
            System.out.print(num + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        printNumbers(1, 2, 3); // Called with multiple arguments
        printNumbers(10);      // Called with a single argument
        printNumbers();        // Called with no arguments
    }
}
```

In this example, the `printNumbers` method uses varargs to accept any number of `int` arguments, which are then iterated and printed.

Characteristics of Varargs:

- **Syntax:**

  A varargs parameter is declared by specifying the data type followed by an ellipsis (...) and the parameter name (e.g., int... numbers).

- **Placement:**

  A varargs parameter must always be the last parameter in a method's parameter list.

- **Single Varargs Parameter:**

  A method can only have one varargs parameter.

- **Internal Representation:**

  The Java compiler internally treats a varargs parameter as an array of the specified type. This means that within the method body, the varargs parameter can be accessed and manipulated like a regular array.

- **Flexibility:**

  Varargs allow a method to be called with zero, one, or multiple arguments of the specified type. If no arguments are passed, the internal array will have a length of zero.

```
import java.io.*;

class Geeks {

    public static void Names(String... n) {

      for (String i : n) {
        System.out.print(i + " ");
      }
      System.out.println();
    }

    public static void main(String[] args) {

      // Calling the 'Names' method with
      // different number of arguments

      Names("A", "AB");
      Names("A", "AB", "ABC");
    }
}
```

USING COMMAND-LINE ARGUMENTS

- **Java command-line argument** is an argument, i.e., passed at the time of running the Java program.
- In Java, the command line arguments passed from the console can be received in the Java program, and they can be used as input.
- The users can pass the arguments during the execution, bypassing the command-line arguments inside the **main()** method.

Why Use Command Line Arguments?

- It is used because it allows us to provide input at runtime without modifying the whole program.
- It helps to run programs automatically by giving them the needed information from outside.

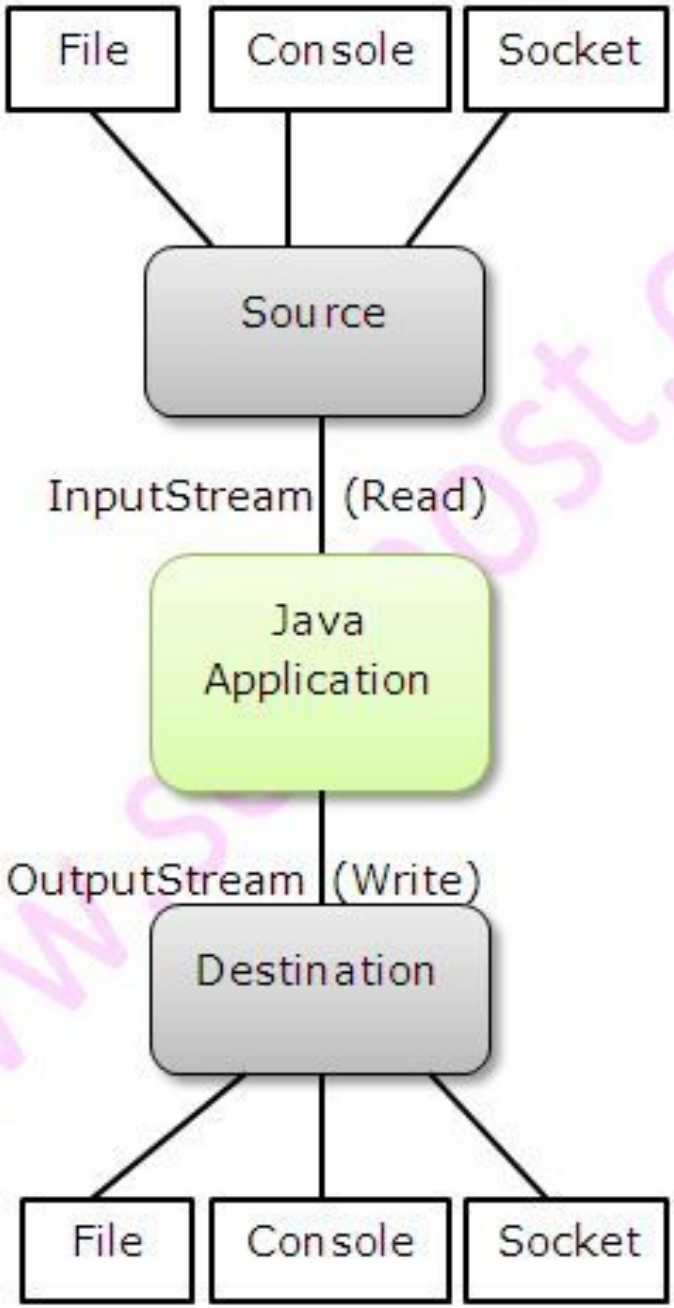
```
class Demo{

    public static void main(String[] args) {

        // Printing the first argument
        System.out.println(args[0]);
    }
}
```

```
C:\Users\HP\Desktop>javac Demo.java

C:\Users\HP\Desktop>java Demo
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
Index 0 out of bounds for length 0
        at Demo.main(Demo.java:6)

C:\Users\HP\Desktop>java Demo "Hi Welcome"
Hi Welcome
```

Managing I/O:

- Streams :Byte Streams and

  Character Streams,
- Predefined Streams,
- Reading console Input,
- Writing Console Output,
- Print Writer class.


File
Console
Socket
Source
InputStream
(Read)
Java
Application
OutputStream
(Write)
Destination
File
Console
Socket

Managing I/O:

- Java provides powerful and flexible ways to handle I/O operations, allowing programs to interact with external data sources and destinations like files, networks, and consoles.
- Managing I/O in Java primarily revolves around the **java.io** package and the concept of **streams**.

- **Streams:** A stream is a sequence of data flowing from a source to a destination.
- **Input Streams:** Used to read data from a source (e.g., keyboard, files).
- **Output Streams:** Used to write data to a destination (e.g., display, files).

Types of I/O Streams in Java:

Byte Streams:

- Handle raw binary data, suitable for files like images or audio. Classes like InputStream and OutputStream are used.
- **Classes**: The fundamental abstract classes for byte streams are InputStream (for reading) and OutputStream (for writing). Concrete implementations include **FileInputStream**, **FileOutputStream**, **ByteArrayInputStream**, **ByteArrayOutputStream**, etc.
- **Mechanism:** They read or write data **one byte** at a time.

- # Character Streams:

- **Purpose:**

  Handle the input and output of 16-bit Unicode characters. They are specifically designed for working with text data, ensuring proper handling of various character encodings and internationalization.

- **Classes:**

  The fundamental abstract classes for character streams are Reader (for reading) and Writer (for writing). Concrete implementations include FileReader, FileWriter, BufferedReader, BufferedWriter, etc.

Mechanism:

- They read or write data character by character, often performing conversions between bytes and characters based on a specified or default character encoding.
- Character streams can be built on top of byte streams, with the character stream layer handling the encoding/decoding. For example, FileReader internally uses FileInputStream.

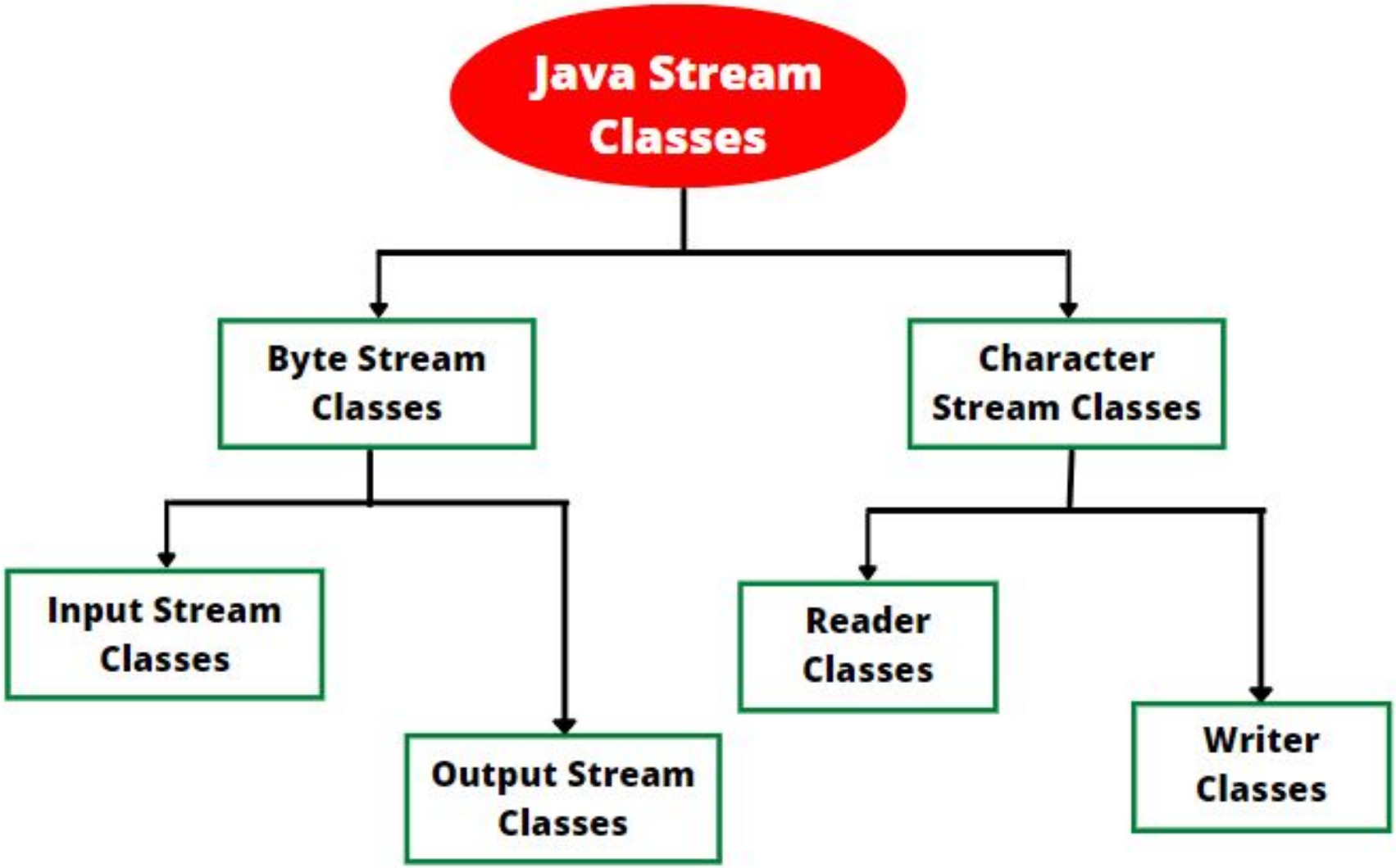


Fig: Classification of Java Stream Classes

PREDEFINED STREAMS IN JAVA

- Java provides three predefined standard streams within the **java.lang.System** class, which are automatically available for use in any Java program without requiring explicit creation or opening.

- These streams facilitate basic input and output operations, typically involving the console.

These predefined streams are:

1.System.in:

This is the standard input stream. It is an instance of InputStream and is typically connected to the keyboard, allowing programs to read data entered by the user.

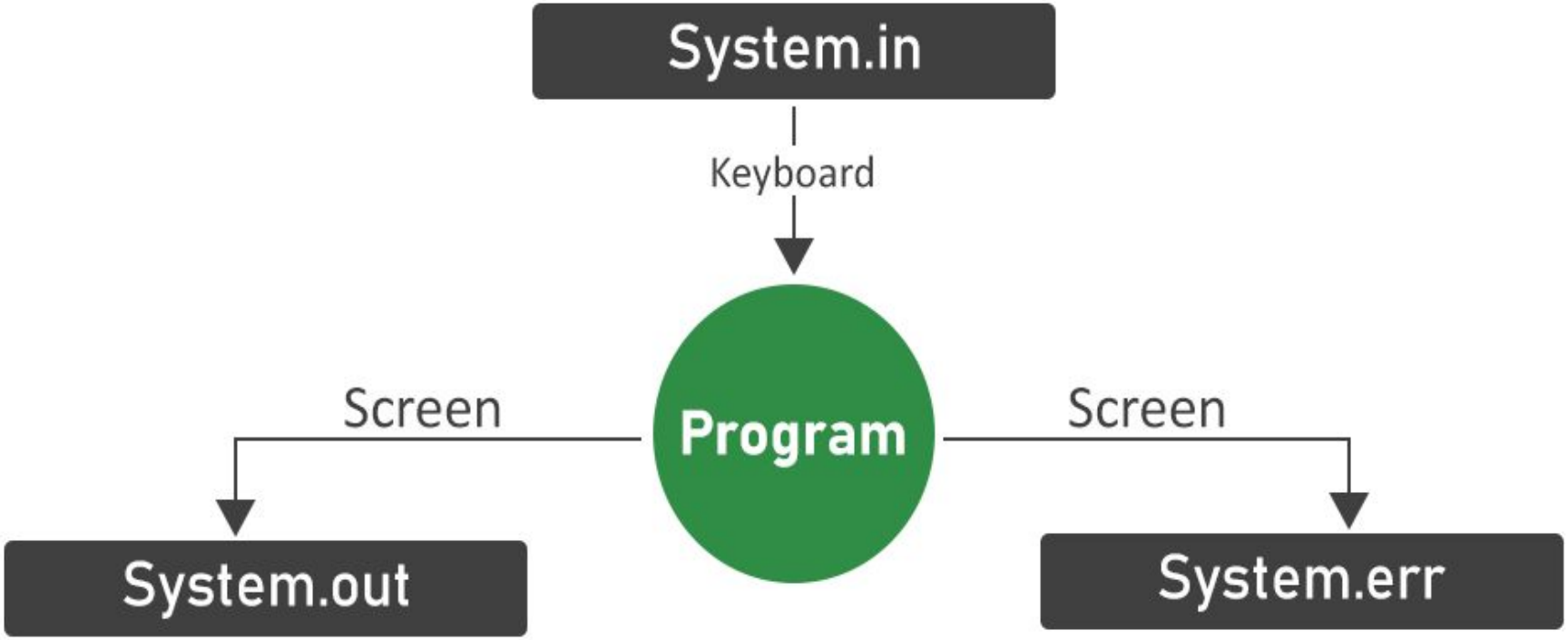
2. System.out:

This is the standard output stream. It is an instance of PrintStream and is typically connected to the console, allowing programs to display text and other data to the user.

3.System.err:

This is the standard error stream. Similar to System.out, it is also an instance of PrintStream and is typically connected to the console.

Standard I/O Streams in Java



Reading console Input in java

Classes provided by Java to take console Input:

There are three classes using which we can take console input:

- **BufferedReader class**
- **Scanner class**
- **Console class**

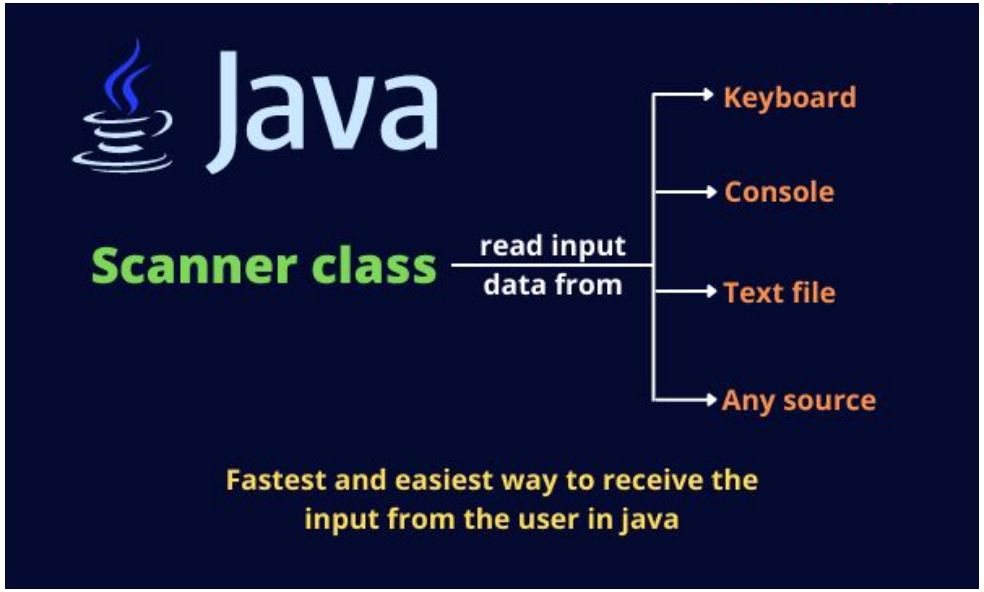
There is also another way to take console input in Java:

- **Using Command Line Argument**

1. Using the Scanner Class:

- The Scanner class, part of java.util, provides methods to parse primitive types and strings from various input sources, including System.in (standard input stream).

```
Scanner sc = new Scanner(System.in);
```



Here's how to use the `Scanner` class to read different types of input:

1. Import the Scanner class:

```
import java.util.Scanner;
```

2. Create a Scanner object:

```
Scanner scanner = new Scanner(System.in);
```

This creates a **`Scanner`** object named `scanner` that is linked to the standard input stream (**`System.in`**), which typically refers to the keyboard.

3. Use appropriate methods to read different data types:

The `Scanner` class offers several methods for reading different data types. Common methods include **`next()` for reading a single token (word) as a `String`**, and **`nextLine()` for reading an entire line of text**.

- You can also read specific data types like

1. `int` with `nextInt()`,
2. `double` with `nextDouble()`,
3. `float` with `nextFloat()`,
4. `boolean` with `nextBoolean()`,
5. `long` with `nextLong()`,
6. `short` with `nextShort()`,
7. `byte` with `nextByte()`

```java
import java.util.Scanner;

public class ScannerInputExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);  // Create a
                                                      Scanner object

        System.out.print("Enter your name: ");
        String name = scanner.nextLine(); // Read a line of text

        System.out.print("Enter your age: ");
        int age = scanner.nextInt(); // Read an integer

        System.out.println("Hello, " + name + "! You are " + age + " years old.");

        scanner.close(); // Close the scanner to release resources
    }
}
```

Output

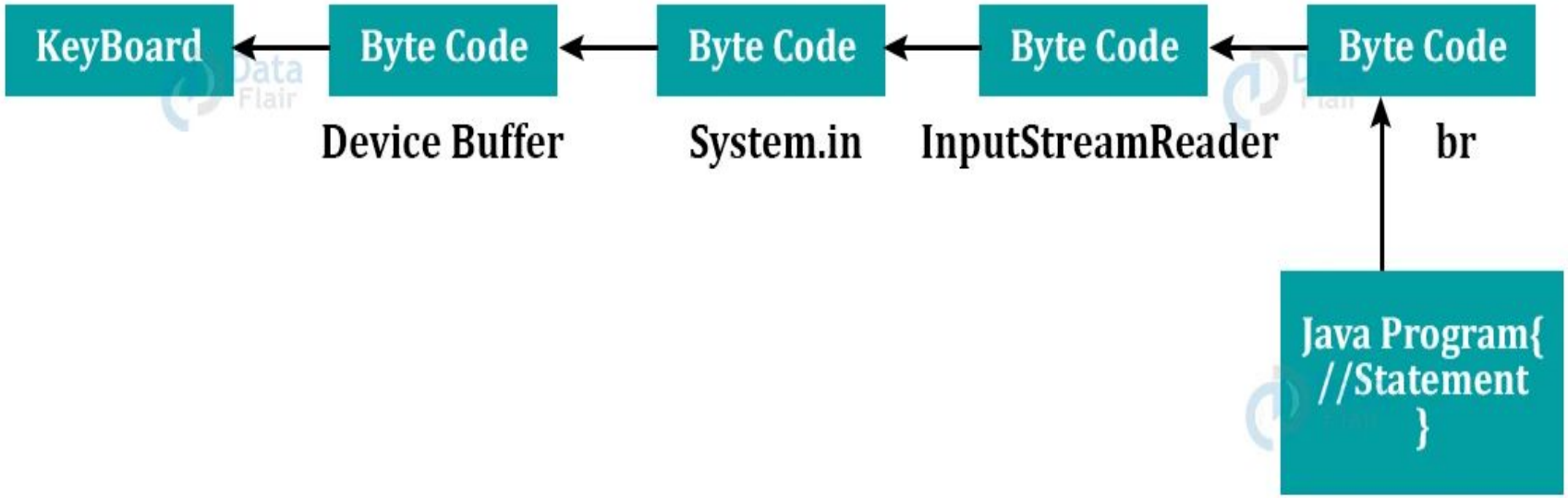
```
Enter your name: Arnav
Enter your age: 10
Hello, Arnav! You are 10 years old.
```

2. Using the BufferedReader Class

- The BufferedReader class, found in java.io.BufferedReader, reads text from a character-input stream, buffering characters to provide efficient reading of lines.
- It is the oldest method introduced in Java to take user input. It has been present in Java since the very beginning. The BufferedReader class wraps the System.in(System Input) with the InputStreamReader.
- Let us see how BufferedReader objects are called:

```
BufferedReader br= new BufferedReader(new InputStreamReader(System.in));
```

How BufferedReader Processes Objects



```java
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class Demo {
   public static void main(String[] args) throws
IOException {
   BufferedReader reader = new BufferedReader(new
                                InputStreamReader(System.in));

      System.out.print("Enter your city: ");
      String city = reader.readLine(); // Read a line of
                                          text

      System.out.println("You live in " + city + ".");

      reader.close(); // Close the reader
   }
}
```

3. Using the Console Class

- The Console class, in java.io.Console, is designed for console-specific interactions, particularly for secure password input as it does not echo characters.

```java
import java.io.Console;

public class ConsoleInputExample {
   public static void main(String[] args) {
       Console console = System.console(); // Get the Console
instance

       if (console != null) {
           char[] password = console.readPassword("Enter
password: "); // Read password without echoing
           String username = console.readLine("Enter username:
");

           System.out.println("Username: " + username);
           System.out.println("Password (hashed): " + new
String(password).hashCode());
       } else {
           System.err.println("No console available.");
       }
   }
}
```

Writing Console Output

- In Java, writing console output is primarily achieved through the System.out object, which is an instance of the PrintStream class.
- The PrintStream class provides several methods for displaying data on the console.

1. print() and println() Methods:

- **System.out.print(data):**

  This method prints the specified data to the console without adding a new line character at the end. Subsequent output will appear on the same line.

- **System.out.println(data):**

  This method prints the specified data to the console and then adds a new line character, moving the cursor to the next line for subsequent output.

```java
public class ConsoleOutputExample {
    public static void main(String[] args) {
        System.out.print("Hello, "); // Prints "Hello, "
        System.out.println("Java!"); // Prints "Java!" and
                                     //moves to a new line
        System.out.println(123);     // Prints the integer 123
                                     //and moves to a new line
        }
}
```

Output:

```
Hello, Java!
123
```

2. write() Method:

While less commonly used for general console output, the write() method of PrintStream allows writing individual bytes to the console.

- System.out.write(int byteval): This method writes the low-order eight bits of the byteval integer as a single byte to the console. This is useful for writing raw byte data or characters represented by their ASCII/Unicode values.

```
public class WriteMethodExample {
   public static void main(String[] args) {
       int charA = 'A';
       System.out.write(charA); // Writes the character 'A'
       System.out.write('\n'); // Writes a newline character
   }
}
```

OUTPUT:

```
A
```

Print Writer class

- The PrintWriter class in Java, found within the java.io package, provides a convenient way to write formatted text to various output destinations, such as files, console, or other Writer or OutputStream objects.
- It is designed for writing human-readable text and offers methods similar to System.out.print(), System.out.println(), and System.out.printf().

Key Features and Characteristics:

Formatted Output:

PrintWriter excels at writing formatted representations of objects and primitive data types as text. It handles the conversion of various data types (integers, floats, strings, objects) into their textual equivalents.

Convenience Methods:

It provides print(), println(), and printf() methods for writing data, with println() adding a line separator and printf() supporting C-style formatted output.

```java
import java.io.File;
import java.io.PrintWriter;
import java.io.FileNotFoundException;

public class PrintWriterExample {
    public static void main(String[] args) {
        try {
            // Create a PrintWriter to write to a file
            PrintWriter writer = new PrintWriter(new File("output.txt"));

            // Write some formatted text
            writer.println("Hello, PrintWriter!");
            writer.printf("The number is: %d%n", 123);

            // Close the writer to ensure all data is flushed
            writer.close();
            System.out.println("Data written to output.txt");
        } catch (FileNotFoundException e) {
            System.err.println("Error: File not found - " + e.getMessage());
        }
    }
}
```

Case Study:

Demonstrate an interactive **Banking/Library management system** using class, objects, array of objects

Important Questions

1. Explain the concept of constructors and the use of this keyword with suitable Java example. [4]
2. Write a Java program to define a class Student with roll, name, and percentage. Use getter and setter methods. [4]
3. Write a Java program to declare and initialize a 2D array. Pass it to a method and find the row-wise sum. [6]
4. Write a Java program to demonstrate use of interfaces and method overriding. [6]
5. Differentiate between primitive and reference data types in Java. [4]
6. What is method overloading? Write a Java program to demonstrate it. [4]
7. Explain command line arguments in Java. Write a program that accepts and displays arguments. [6]

1. State the difference between **instance** and **static** variables.
2. What does method overloading mean? Give an example.
3. List three access specifiers in Java and explain their scope.
4. Explain passing an object as a parameter to a method.
5. Define byte stream and character stream in Java I/O.
6. What are command-line arguments in Java?
7. Write a Java program defining a Person class with name/age, including constructors, getters/setters.
8. Demonstrate method overloading in a class for two methods named add()—one for adding ints, another for doubles.
9. Explain with example: static method, static variable, and when to use them.
10. Describe how to return an object from a method, with code.
11. Write a snippet illustrating multidimensional arrays in Java and passing them to a method.
12. Show how to use variable-length argument lists (...) in a method.
13. Write code to read console input using Scanner, then print a formatted output using PrintWriter.

1. **Banking System**: Design a BankAccount class with fields (name, accNo, balance), methods: deposit, withdraw (with balance check), display. Include constructors and illustrate usage in main.

2. **Library System**: Create a Book class (title, author, ISBN, isIssued). Use an array of Book objects, and write methods to issue and return books.

3. Write a Java program that accepts n integers as command-line arguments, stores them in an array, sorts them, and prints the result.

4. Implement **constructor chaining** using this(...) in a class with multiple constructors.

5. Explain the use of **PrintWriter** for console output and compare it with System.out.println(), with code.

Thank You!!!